Programación 1 — Unidad 9: Manejo de archivos

Material complementario: archivos CSV (listas y diccionarios)

Índice

Archivos CSV en Python

– ¿Qué es un archivo CSV?

– Lectura de archivos CSV con Python

– Lectura con csv.DictReader (uso de diccionarios)

– Escritura de archivos CSV



¿Qué es un archivo CSV?

Un archivo **CSV** (*Comma Separated Values* o Valores Separados por Comas) es un tipo de archivo de texto plano diseñado para almacenar datos en forma de tabla. Aunque al abrirlo con un editor de notas parece simple texto, su estructura interna sigue reglas muy precisas que permiten organizar grandes volúmenes de información de manera eficiente.

Relación con las planillas de cálculo (Excel, Sheets)

La mejor forma de entender un CSV es compararlo con una planilla de cálculo como **Microsoft Excel** o **Google Sheets**:

- **Líneas como filas**: cada línea de texto en el archivo representa una **fila** completa de la tabla.
- **Comas como celdas**: cada elemento separado por una coma funciona como una **celda** individual.

- **Encabezados**: generalmente, la primera línea del archivo contiene los nombres de las columnas, definiendo qué representa cada «celda» posterior (por ejemplo: `nombre,apellido,email`).

Debido a esta estructura, casi todos los programas de hojas de cálculo pueden abrir un CSV y mostrarlo automáticamente en su formato de celdas tradicional, lo que facilita enormemente la visualización y edición manual de los datos.

¿Para qué se usan?

En el desarrollo de software y la ciencia de datos, los archivos CSV son fundamentales por varias razones:

- **Interoperabilidad**: es un lenguaje «universal». Un sistema escrito en Java puede exportar un CSV que luego será procesado por un script en Python sin problemas de compatibilidad.
- **Persistencia**: permiten guardar los resultados de un programa (datos en RAM) de forma permanente en el almacenamiento del disco.
- **Ligereza**: al ser texto plano sin formatos complejos (como negritas, colores o fórmulas), ocupan mucho menos espacio que un archivo `.xlsx` de Excel.
- **Entrada de datos**: son ideales para cargar listados masivos, configuraciones o registros históricos en una base de datos.

Nota

Aunque el nombre indica «comas», en muchos países (como Argentina) es común usar el **punto y coma (`;`)** como separador, ya que la coma se reserva para los decimales. En Python, aprenderemos que podemos adaptar nuestro código a cualquiera de estos casos usando el parámetro `delimiter`.

Lectura de archivos CSV con Python

Para procesar estos archivos, Python no los trata como un simple bloque de texto, sino que los interpreta fila por fila para transformarlos en objetos manipulables (como listas o diccionarios).

La importación del módulo

Lo primero es incluir `import csv` al inicio de nuestro script. Este es un módulo de la **biblioteca estándar**, lo que significa que ya viene instalado con Python; no necesitás descargar nada extra, pero debés importarlo para acceder a sus herramientas específicas de parseo (análisis de datos).

El parámetro encoding="utf-8" en open()

Al abrir el archivo con la función `open()`, es fundamental incluir el parámetro **encoding**.

- **¿Qué es?** Es el «traductor» que le dice a Python cómo interpretar los bytes del archivo para que se vean como letras.

- **¿Por qué usar `"utf-8"`?** Es el estándar universal. Si tu archivo tiene **tildes, eñes o caracteres especiales** y no usás el encoding correcto, el programa lanzará un error o mostrará símbolos extraños.

El procedimiento de apertura (open)

Antes de leer, debemos abrir el canal de comunicación con el archivo:

```
with open("datos.csv", "r", encoding="utf-8") as archivo:
    # Aquí va el resto del código
```

El objeto csv.reader

Al ejecutar `lector = csv.reader(archivo)`, Python crea un objeto especial llamado **iterador**.

- **¿Lee todo de golpe?** No. Esta es una gran ventaja: el `reader` va procesando el archivo línea por línea a medida que lo recorremos. Esto permite procesar archivos gigantes (de gigabytes) sin agotar la memoria RAM de la computadora.
- **¿Cómo toma cada fila?** El `reader` interpreta cada línea del texto y la transforma automáticamente en una **lista de Python**. Cada «celda» del CSV se convierte en un elemento de esa lista.

El manejo de los encabezados

Por defecto, `csv.reader` no sabe distinguir si la primera fila son datos o son los títulos de las columnas (*headers*). Si simplemente hacemos un `for`, la primera vuelta imprimirá los títulos. Para «saltarnos» los encabezados y trabajar solo con los datos, usamos la función `next()`:

```
import csv

with open("alumnos.csv", "r", encoding="utf-8") as archivo:
    lector = csv.reader(archivo)

    # Extraemos la primera fila (encabezados) y movemos el
    # puntero a la segunda
    encabezados = next(lector)
    print(f"Columnas detectadas: {encabezados}")

    # Ahora el bucle empieza directamente en los datos
    for fila in lector:
        # fila es una lista, ej: ["Juan", "20", "8.5"]
        nombre = fila[0]
        nota = fila[2]
        print(f"Alumno: {nombre} - Calificación: {nota}")
```

El parámetro delimiter

No todos los archivos CSV usan comas. En regiones donde la coma se usa para decimales (como Argentina), es muy común encontrar archivos separados por punto y coma `;`. Cómo manejarlo:

```
with open("datos.csv", "r") as archivo:
    lector = csv.reader(archivo, delimiter=";")
    # ... resto del código
```

Lectura con csv.DictReader (uso de diccionarios)

A diferencia del `reader` común que nos devuelve listas, el **DictReader** mapea la información de cada fila directamente a un **diccionario de Python**.

¿Cómo funciona la relación columna-clave?

El `DictReader` asume por defecto que la **primera línea** del archivo contiene los nombres de las columnas (encabezados).

- Utiliza esos nombres como **claves** (*keys*).
- Utiliza el contenido de cada celda como **valor** (*value*).

Ejemplo práctico de implementación

Imaginemos un archivo llamado `productos.csv` con este contenido:

```
id,nombre,precio
1,Teclado,5000
2,Mouse,3000
```

```
with open("productos.csv", "r", encoding="utf-8") as archivo:
    # Creamos el lector de diccionarios
    lector_dict = csv.DictReader(archivo)

    for fila in lector_dict:
        # 'fila' ahora es un diccionario:
        # {'id': '1', 'nombre': 'Teclado', 'precio': '5000'}
        print(f"Producto: {fila['nombre']} - Precio: ${fila['precio']}")
```

Ventajas clave

Buenas prácticas

1. Legibilidad (*readability*): es mucho más claro escribir `fila['precio']` que recordar que el precio estaba en la posición `fila[2]`.

2. Mantenimiento: si el día de mañana el archivo CSV cambia el orden de las columnas (por ejemplo, el precio pasa a estar primero), el código **seguirá funcionando** porque busca por nombre de clave, no por posición.

3. No requiere `next()`: a diferencia del `reader` común, el `DictReader` ya sabe que la primera fila es el encabezado y la consume automáticamente para armar las claves. No necesitamos saltarla manualmente.

¿Qué pasa si el CSV no tiene encabezados?

Si el archivo empieza directamente con los datos, podemos pasarle los nombres de las columnas manualmente mediante el parámetro `fieldnames`:

```
# Le decimos nosotros cuáles son los nombres de las columnas
columnas = ["id", "nombre", "precio"]

with open("datos_sin_titulo.csv", "r", encoding="utf-8") as archivo:
    lector = csv.DictReader(archivo, fieldnames=columnas)

    for fila in lector:
        print(fila["nombre"])
```

Comparación: reader vs. DictReader

Característica	csv.reader	csv.DictReader
Formato de salida	Lista (`list`)	Diccionario (`dict`)
Acceso a datos	Por índice: `fila[0]`	Por nombre: `fila['nombre']`
Encabezados	Deben saltarse con `next()`	Se usan automáticamente como claves
Ideal para…	Archivos simples o sin títulos	Archivos estructurados y código limpio

Escritura de archivos CSV

Escribir un archivo CSV implica transformar estructuras de datos de Python (como listas o diccionarios) en una cadena de texto donde los elementos estén separados por comas y finalicen con un salto de línea.

Parámetros críticos en el open para escritura

Cuando abrimos un archivo para escribir (`mode="w"` o `mode="a"`), debemos sumar un parámetro extra:

```
with open("archivo.csv", "w", newline="", encoding="utf-8") as archivo:
```

- `mode="w"`: abre el archivo para escritura. **Cuidado**: si el archivo ya existe, se borrará todo su contenido antes de empezar. Si no existe, se creará uno nuevo.
- `newline=""`: este es el parámetro más importante en Windows. Si no se incluye, Python podría agregar saltos de línea extra (filas vacías) entre cada dato, corrompiendo el formato del CSV.
- `encoding="utf-8"`: asegura que los caracteres especiales se guarden correctamente en el disco.

Escritura con csv.writer (basada en listas)

Se utiliza cuando nuestros datos están organizados en una lista de listas o una lista de tuplas.

```
# Datos en la RAM (lista de listas)
alumnos = [
    ["Nombre", "Materia", "Nota"],
    ["Matias", "Programación", "9"],
    ["Lucía", "Sistemas", "10"],
    ["Jorge", "Bases de Datos", "7"]
]

with open("notas_alumnos.csv", "w", newline="", encoding="utf-8") as archivo:
    # Creamos el objeto escritor
    escritor = csv.writer(archivo)

    # Opción 1: escribir todas las filas de golpe
    escritor.writerows(alumnos)

    # Opción 2: escribir una sola fila (útil dentro de un bucle)
    # escritor.writerow(["Nuevo Alumno", "Materia", "10"])
```

- `writerow(lista)`: recibe una lista y la escribe como una sola línea en el archivo.
- `writerows(lista_de_listas)`: recibe una estructura completa y la vuelca entera al archivo.

Escritura con csv.DictWriter (basada en diccionarios)

Es la forma más segura y estructurada. Obliga al programador a definir primero cuáles serán los encabezados (las columnas) del archivo.

```
# Datos en la RAM (lista de diccionarios)
empleados = [
    {"id": 101, "nombre": "Roberto", "puesto": "Desarrollador"},
    {"id": 102, "nombre": "Elena", "puesto": "Analista"},
    {"id": 103, "nombre": "Marcos", "puesto": "Tester"}
]

# 1. Definimos el orden de las columnas (claves del diccionario)
columnas = ["id", "nombre", "puesto"]

with open("nomina.csv", "w", newline="", encoding="utf-8") as archivo:
    # 2. Creamos el escritor indicando los nombres de las columnas
    escritor_dict = csv.DictWriter(archivo, fieldnames=columnas)

    # 3. ¡Importante! Escribir la primera fila de títulos
    escritor_dict.writeheader()

    # 4. Volcamos los datos
    escritor_dict.writerows(empleados)
```

¿Por qué usar DictWriter?

Qué hace el código:

- **Seguridad**: si un diccionario tiene una clave que no está definida en `fieldnames`, Python lanzará un error, evitando que guardes datos en columnas inexistentes.
- **Independencia del orden**: no importa en qué orden estén las claves dentro de tus diccionarios de Python; `DictWriter` las organizará siempre según el orden que definiste en `fieldnames`.
- **Encabezados automáticos**: el método `writeheader()` genera automáticamente la fila de títulos basada en tus nombres de columnas.